

# Fitness In Service

## 혼자 기획하고 운영하는 헬스장 통합 관리 플랫폼

요구사항 정의 · DB 설계 · API 개발 · 배포 · 운영까지 전 과정을 직접 수행하며, 어떻게 설계하고 무엇을 근거로 결정하는지를 보여주는 프로젝트입니다.

Java 17 · Spring Boot 4

MyBatis · MariaDB · Redis

AWS · Docker

Toss · JWT · SSE

### WHY

## 이 프로젝트를 시작한 이유

커머스 백엔드로 일하며 상품·주문·프로모션 등 주어진 도메인의 일부를 깊게 다뤄왔습니다. 다만 실무에서는 이미 만들어진 시스템 위에서 일부를 맡는 경우가 많아, 서비스 하나를 요구사항부터 운영까지 통째로 책임지는 경험을 스스로 만들고 싶었습니다.

그래서 실제 운영 니즈가 있는 도메인(헬스장 운영 — 센터·회원·PT세션·급여·결제)을 골라, 관리자 백오피스와 회원 앱을 아우르는 백엔드를 처음부터 설계했습니다. 동시에 **AI 에이전트를 개발 프로세스에 정식으로 결합**했을 때 1인 개발이 어디까지 가능한지를 실험하는 목적도 있었습니다.

🔗 계기 초안입니다 — 실제 시작 계기(예: 지인 헬스장 니즈, 특정 문제 해결, 도메인 선택 이유 등)를 한두 문장 주시면 사실 그대로 교체하겠습니다.

## PROCESS

# 일하는 방식 — 기획부터 배포까지

단계를 나눠 진행하고, 각 단계의 규칙과 결정을 문서로 남깁니다.



**규칙 우선** 모든 작업 전 설계 규칙·프로젝트 메모리 문서를 먼저 참조해 컨텍스트를 고정

**문서 동기화** API 스펙이 바뀌면 정의서를 표준 절차로 함께 갱신

**품질 게이트** 커밋 전 해당 모듈 빌드 검증, 실패 시 커밋 금지

**결정 기록** 도메인별 결정·근거를 메모리 문서에 누적해 재현 가능하게 유지

## DESIGN

# 설계 원칙

### DDD 기반 멀티모듈

공유 도메인은 **Core**, 관리자(BO)·회원(MO)은 별도 모듈. 공유 코드는 반드시 Core에 두어 경계를 강제.

### 계층 & 경계 분리

Controller → Service → Mapper → Entity. **DTO** 는 API 계층, **VO** 는 Service↔Mapper — 도메인이 DTO에 의존하지 않게.

### 응답·예외 통일

모든 응답은 **ApiResponse<T>**, 예외는 **BusinessException** +에러코드 enum으로 일관되게 처리.

### 정합성·추적성 우선

상수 중앙화로 매직리터럴 제거, Traceld 필터+AOP로 요청 전 구간 로깅해 장애를 추적 가능하게.

## 어떻게 결정하는가 — 실제 판단 사례

상황 · 판단 · 근거. 코드 레벨 상세는 면접에서 설명 가능합니다.

### 결제 중복 처리를 구조로 막을까, 사후 보정할까

- 상황** 승인 요청과 Toss Webhook이 동시에 도착하거나 재시도가 겹치면 같은 결제가 중복 반영될 수 있음.
- 판단** 사후 보정 대신 구조로 원천 차단. 비관적 락( `FOR UPDATE` )+먹등키로 진입 자체를 1회로 제한, 환불은 잔여 환불가능액 기준.
- 근거** 돈이 걸린 도메인은 "고쳐서 맞추기"보다 "틀릴 수 없게 만들기"가 운영 리스크를 줄인다고 판단.

### GX 예약 취소를 회원 앱에도 열까

- 상황** 회원 앱에서 직접 취소를 허용할지, 관리자만 처리할지 결정 필요. BO·MO가 같은 DB를 공유.
- 판단** 취소는 관리자(BO)에서만 처리하도록 확정하고 회원 앱은 조회만.
- 근거** 취소는 세션·정산·알림에 연쇄 영향이 커, 통제 지점을 하나로 모아 상태 일관성을 지키는 쪽을 택함.

### 운영 중인 통합조회 쿼리를 고칠까, 새로 만들까

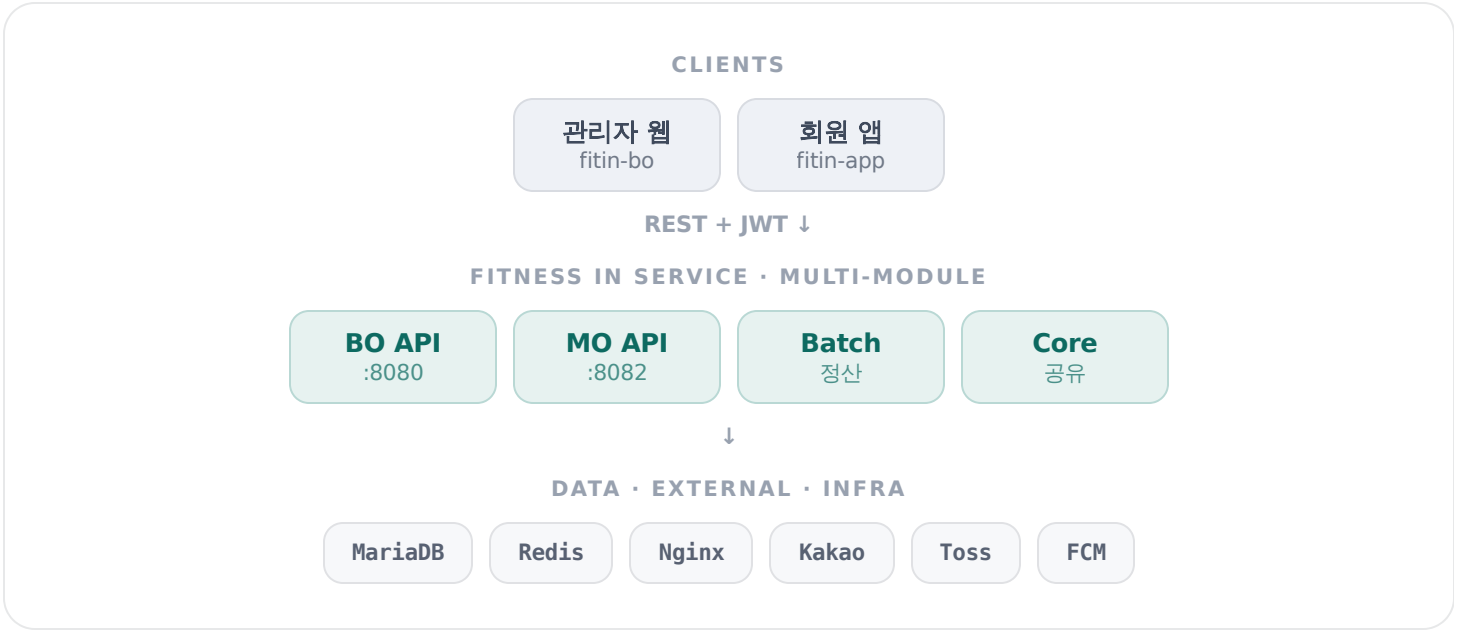
- 상황** 도메인별 목록 API가 필요했는데, 기존 통합조회 쿼리는 여러 화면이 함께 쓰는 자산.
- 판단** 기존 쿼리는 손대지 않고, 단일 파라미터 기반으로 도메인별 신규 쿼리를 각 Mapper에 분리 작성.
- 근거** 운영 자산의 사이드이펙트를 원천 차단 — 확장은 격리해서, 기존 동작은 그대로 보존.

### 집계 버그를 어떻게 안전하게 고칠까

- 상황** 상태 문자열 오타( `RESERVE` )로 예약중 세션이 소진으로 잘못 집계 → 급여 오차.
- 판단** 고치기 전 급여정산의 화이트리스트(COMPLETED/NO\_SHOW만 인정)와 대조해 의도를 검증하고, 오타 전파 여부를 전수 검색.
- 근거** "정상 범위를 좁게 정의"하는 화이트리스트가 블랙리스트보다 실수에 강하다고 보고 기준으로 삼음.

ARCHITECTURE

아키텍처



STACK

기술 스택 & 규모

|         |         |               |               |              |         |       |
|---------|---------|---------------|---------------|--------------|---------|-------|
| Backend | Java 17 | Spring Boot 4 | MyBatis       | Spring Batch |         |       |
| Data    | MariaDB | AWS RDS       | Redis         | Pub/Sub      |         |       |
| 인증·결제   | JWT     | Kakao OAuth   | Toss Payments | Webhook      |         |       |
| 실시간·인프라 | SSE     | STOMP         | FCM           | Docker       | AWS EC2 | Nginx |

70 도메인 엔티티    4 Gradle 모듈    3 저장소 (백엔드·관리자·회원앱)    1,700+ 누적 커밋

beta.fitness-trainer-assistant.ai.kr

실제 배포된 베타 서비스 · 직접 확인 가능

서비스 열기 ↗

소스코드는 실서비스 운영 중인 관계로 비공개 · 설계·의사결정의 코드 레벨 상세는 면접에서 설명 가능 · 2026.07